

Round 1: Multiple Choice Showdown

1. What is the output, x?

```
int x = 6;
int *p = &x;
int *q = p;
*q += 4;
p = NULL;
cout << x;
```

- A. 6
- B. 10
- C. 0
- D. Undefined behavior

2.

```
char* tag = static_cast<char*>(malloc(4 * sizeof(*tag)));

tag[0] = 'C';
tag[1] = 'O';
tag[2] = 'D';
tag[3] = 'E';

cout << tag << endl;
```

- A. Always prints CODE
- B. Always prints CODE followed by one space
- C. Behavior is undefined because no null terminator is stored
- D. malloc adds the terminator automatically

3.

```
// Request a larger memory block using a temporary pointer
int* temp = static_cast<int*>(
    realloc(data, newCapacity * sizeof(*data))
);

// Check whether realloc() failed
if (temp == nullptr)
```

```
{  
    /* ? */  
}
```

- A. free(data); capacity=newCap;
- B. data=tmp; capacity=newCap;
- C. Return failure without changing data or capacity
- D. Set data=NULL and continue

4.

```
// Shift elements one position to the right  
for (int i = size; i > index; --i)  
{  
    data[i] = data[i - 1];  
}  
  
// Insert the new value  
data[index] = value;  
  
// Update the logical size  
++size;
```

- A. Correct after capacity and index validation
- B. Must use $i \geq \text{index}$
- C. Must start at size-1
- D. Must shift left

5.

```
int main()  
{  
    int data[8] = {3, 8, 13, 21};  
  
    int size = 4;  
    int capacity = 8;  
    int hits = 0;  
  
    for (int i = 0; i < capacity; ++i)  
    {
```

```

        if (data[i] == 0)
        {
            ++hits;
        }
    }

    cout << "hits = " << hits << endl;

    return 0;
}

```

- A. hits is evidence about four stored values
- B. hits is 4, but the loop incorrectly treats unused slots as logical data
- C. hits is 0 because size is 4
- D. The loop writes beyond capacity

6.

```

int* data = static_cast<int*>(
    malloc(3 * sizeof(*data))
);

if (data == nullptr)
{
    cerr << "Memory allocation failed." << endl;
    return 1;
}

data[0] = 10;
data[1] = 20;
data[2] = 30;

// alias points inside the same allocation
int* alias = &data[1];

cout << "Before free:" << endl;
cout << "data[1] = " << data[1] << endl;
cout << "*alias = " << *alias << endl;

free(data);

```

- A. alias remains valid because it is a different pointer
- B. alias becomes NULL automatically
- C. alias is dangling and must not be dereferenced
- D. only data[0] is released

Round 2: Blank Space Surgery

#	Incomplete code
R2.1	void setZero(int ____ p) { *p = 0; }
R2.2	Student *p = &s; cout << p____score;
R2.3	FILE *fp = fopen(name, "r"); if (fp == ____) return false;
R2.4	int *tmp = (int*)realloc(data, newCap * ____);
R2.5	for (int i = size; i > index; ____ i) a[i] = a[i-1];
R2.6	for (int i = index; i < size - 1; ++i) a[i] = a[i ____ 1];
R2.7	free(data); data = ____;
R2.8	if (parsedFields == expected && valid) { a[size] = temp; ____size; }

Round 3: Clue Match Duel

Set 1

No	Item
1	A function overwrites its only owner pointer with a new address without freeing the old allocation.
2	After free(records), an alias still holds &records[2].
3	At every public operation boundary, $0 \leq \text{size} \ \&\& \ \text{size} \leq \text{capacity}$ must be true.

4	A loader parses into temporary fields, rejects malformed/duplicate data, then stores once and increments size.
5	After sorting, each student ID remains attached to that student's name and score.
6	An insertion writes a[size] when size == capacity and growth was not performed.

Options

Option	Meaning
A	Memory Leak
B	Dangling Pointer
C	Invariant
D	Read – Validate – Commit
E	Whole-Record Swap
F	Buffer Overrun

Set 2: Records and File Processing

No.	Item	Answer
1	Malformed record	_____
2	Duplicate ID	_____

No.	Item	Answer
3	Whole-record swap	_____
4	Commit	_____
5	Checked save	_____

Options

Option	Meaning
A	Publish one fully validated record and then increase size
B	Confirm every formatted write and the final close
C	Input missing required fields or containing unexpected extra data
D	Move every field together during sorting
E	A key already stored in the collection